Khushi Chhaproo | Div.: D20C | Roll No.: 07

Blockchain Experiment 7

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connect Ganache with Metamask and Remix IDE:

1. Install Ganache:
   - ➔ Download and install Ganache from the Truffle Suite website.
   - ➔ Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
   - ➔ Open Metamask and switch to a Custom RPC Network.
   - ➔ Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
   - ➔ Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
   - ➔ Open Remix IDE in the browser. Choose Injected Web3 as the environment in the "Deploy & Run Transactions" tab.
   - ➔ Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
   - ➔ Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.). pragma solidity ^0.8.0; contract SimpleStorage { uint public data;

function
set(uint x) public { data = x; } 5. Compile and Deploy the Contract: Compile using Remix.

➔ Deploy via the Metamask account connected to Ganache.

➔ Approve the transaction in Metamask.

6. Check Transaction Details on Ganache:

➔ View mined transactions, block details, and balances in the Ganache interface.

7. Interact with the Smart Contract:

➔ Use Remix to call contract functions (set() or get()).

➔ Observe the state changes and corresponding transactions on Ganache.

Outcome:

- Understood how to link Metamask with Remix IDE for Ethereum smart contract deployment.
- Successfully deployed and interacted with a Solidity smart contract through Ganache.
- Verified all transactions and balances within the Ganache environment.

Program Code:

BugBountyEscrow.sol

```
/// SPDX-License-Identifier: MIT pragma
solidity ^0.8.0;

contract BugBountyEscrow {
    enum Status { Created, Submitted, Approved,
Paid, Refunded }

    struct Bounty {
        uint256 bountyId;
            address payable company;
              address payable hunter;
        uint256 amount;
        string bugReportHash;
        Status status;
    }

    uint256 public bountyCount;
    mapping(uint256 => Bounty) public bounties;

    event BountyCreated(uint256
indexed bountyId, address indexed company,
uint256 amount);
    event BugSubmitted(uint256 indexed bountyId,
address indexed hunter, string bugReportHash);
        event BountyApproved(uint256
indexed bountyId);
    event RewardPaid(uint256 indexed bountyId,
address indexed hunter, uint256 amount);

    function createBounty() external payable {
        require(msg.value > 0, "Bounty reward
must be greater than zero");

        bountyCount++;
            bounties[bountyCount] =
Bounty({
                bountyId: bountyCount,
            company:
payable(msg.sender),
            hunter:
payable(address(0)),
                amount: msg.value,
            bugReportHash: "",
                status: Status.Created
        });
```

```
        emit
BountyCreated(bountyCount, msg.sender, msg.value);
    }

    function submitBug(uint256
_bountyId, string memory
_bugReportHash) external {
            Bounty storage bounty =
bounties[_bountyId];
        require(bounty.bountyId != 0, "Bounty
does not exist");
            require(bounty.status ==
Status.Created, "Bounty is not open for
submission");

        bounty.hunter =
payable(msg.sender);
        bounty.bugReportHash =
_bugReportHash;
        bounty.status =
Status.Submitted;

                emit BugSubmitted(_bountyId,
msg.sender, _bugReportHash);
    }

    function approveBug(uint256
_bountyId) external {
            Bounty storage bounty =
bounties[_bountyId];
        require(msg.sender ==
bounty.company, "Only company can
approve");
            require(bounty.status ==
Status.Submitted, "No submission to approve");

        bounty.status =
Status.Approved;
        emit
BountyApproved(_bountyId);
    }

    function releasePayment(uint256 _bountyId)
external {
            Bounty storage bounty =
bounties[_bountyId];
        require(msg.sender ==
bounty.company, "Only company can release
payment");
            require(bounty.status ==
Status.Approved, "Bounty report not approved
yet");

                bounty.status = Status.Paid;
            uint256 paymentAmount =
bounty.amount;
        bounty.amount = 0;


  bounty.hunter.transfer(paymentAmount); emit
            RewardPaid(_bountyId,
bounty.hunter, paymentAmount); }

    function getBounty(uint256
_bountyId) external view returns ( uint256
                bountyId,
        address company,
        address hunter,
        uint256 amount,
                string memory bugReportHash,
        Status status
    ) {
        Bounty memory b =
bounties[_bountyId];
            require(b.bountyId != 0,
"Bounty does not exist");
        return (b.bountyId, b.company, b.hunter,
b.amount, b.bugReportHash, b.status);
    }
}
```

Implementation:

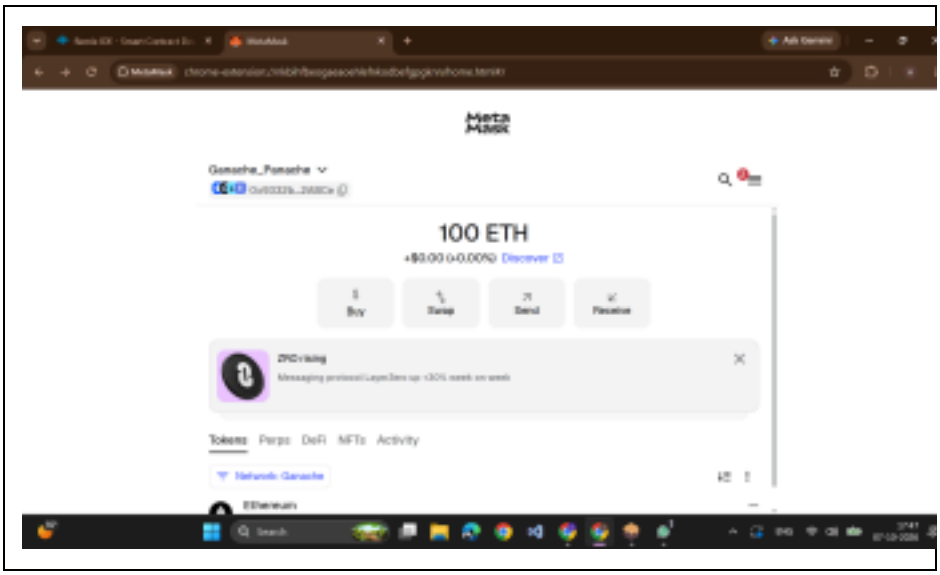
Step 1: Environment Setup & Ganache Configuration

1. Launch **Ganache Desktop GUI** and select **Quickstart** to establish a local Ethereum

blockchain network.

2. Verify that the local RPC Server is running on HTTP://127.0.0.1:7545 with Network ID / Chain ID set to 1337 / 5777.

3. Open **Remix IDE** in the browser and navigate to the **Deploy & Run Transactions** tab. 4. Change the **Environment** dropdown to **Custom - External HTTP Provider** and set the endpoint URL to http://127.0.0.1:7545 to connect Remix directly to Ganache.



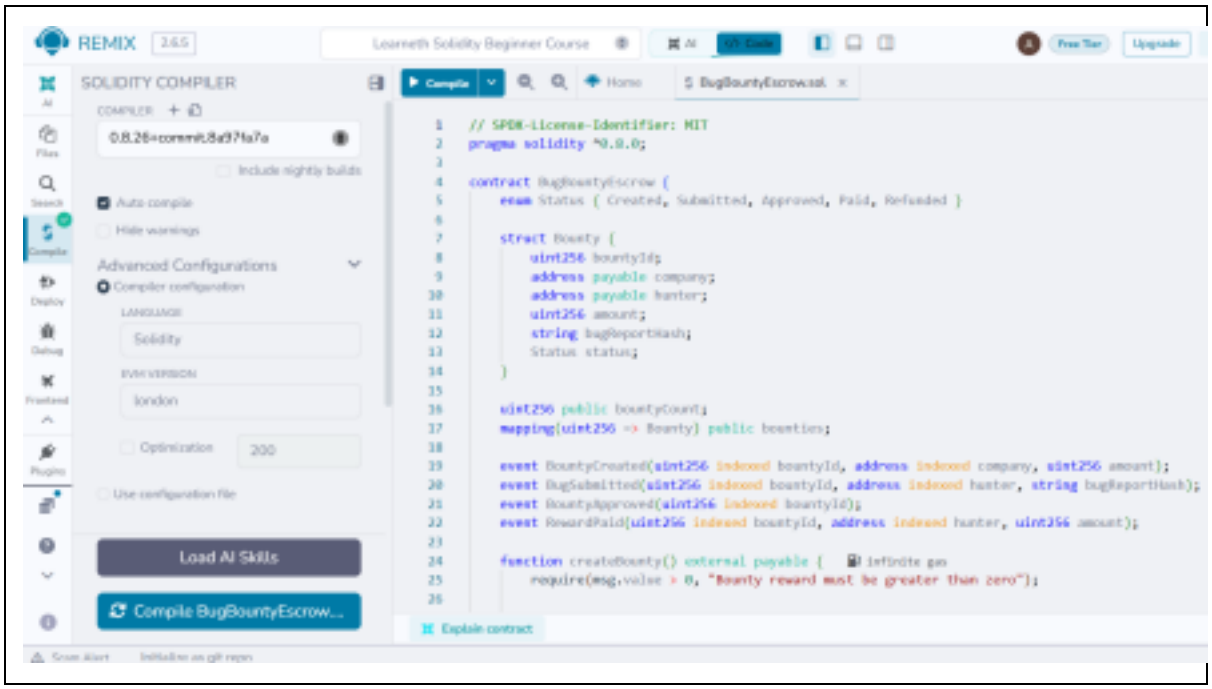
Step 2: Smart Contract Creation & Compilation

1. In the Remix File Explorer, create a new file named BugBountyEscrow.sol. 2. Write the Solidity smart contract code implementing the escrow logic, struct data structures, events, and functions (createBounty, submitBug, approveBug, releasePayment).

3. Open the **Solidity Compiler** tab, set the compiler version to 0.8.26, and click **Compile BugBountyEscrow.sol**.

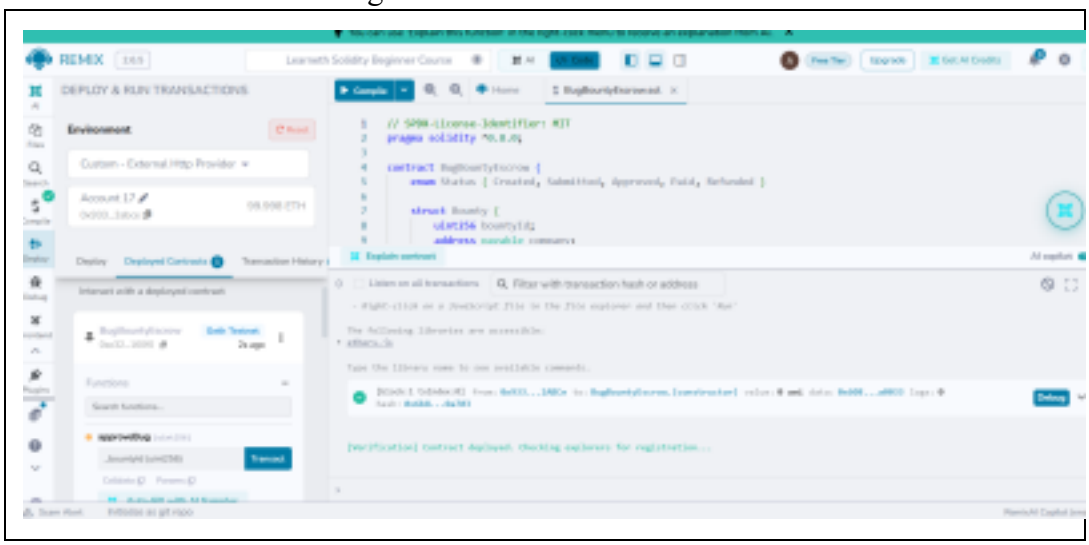
Step 3: Contract Deployment

1. Return to the **Deploy & Run Transactions** tab.
2. Select an account from the **Account** dropdown (pre-funded with 100 ETH from Ganache).
3. Ensure **Value** is set to 0 and **Gas limit** is set to 3000000.
4. Click **Deploy**.
5. Verify that the transaction is mined on Ganache and the deployed contract instance appears under **Deployed Contracts**.



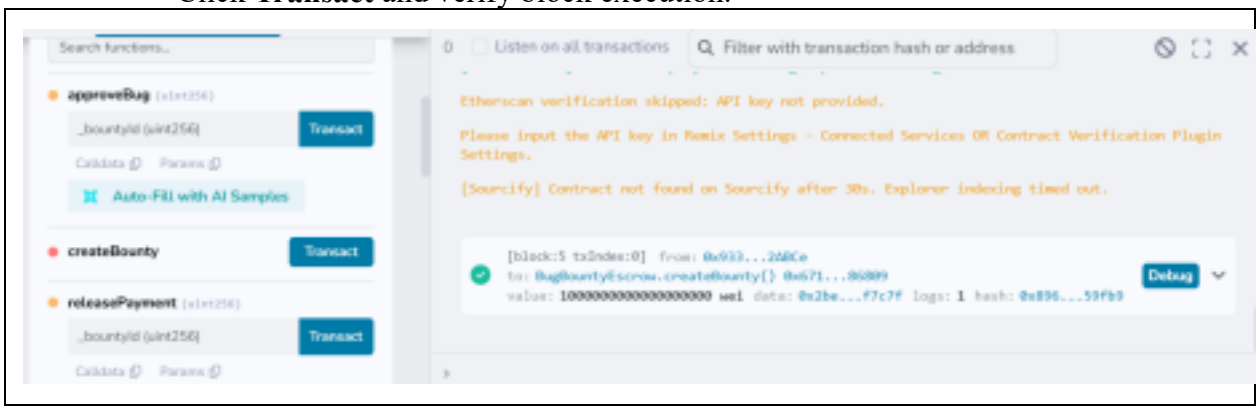
Step 4: Executing Contract Functions (Lifecycle Execution)

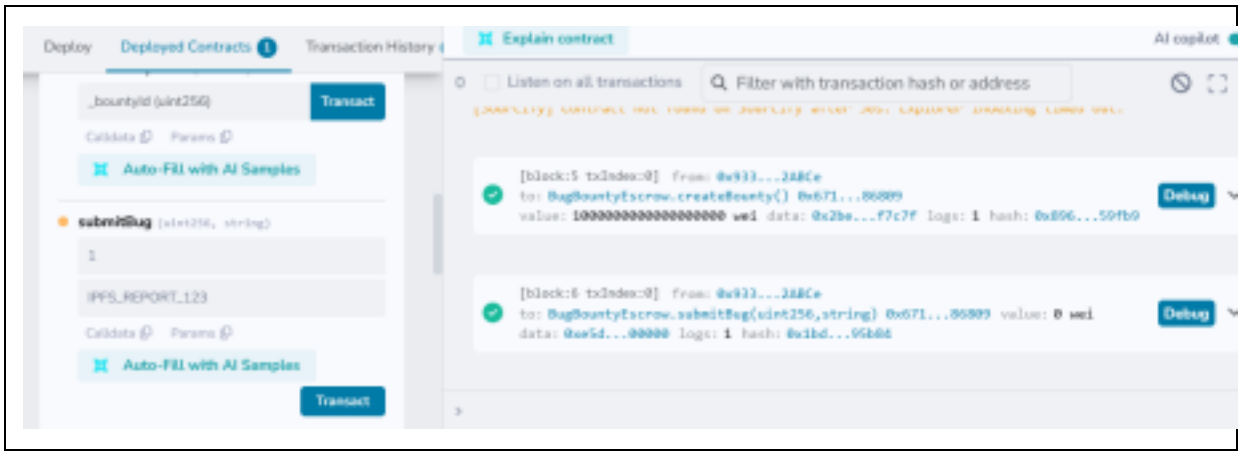
1. **createBounty**:
   - Set **Value** to 1 and unit dropdown to **Ether**.
   - Under **Deployed Contracts** → BugBountyEscrow, click **createBounty**. ○ Confirm that 1 ETH is transferred into the contract and the terminal displays block execution with a green checkmark.



2. **submitBug**:
   - Reset **Value** back to 0.

○ Expand **submitBug**, enter _bountyId: 1 and _bugReportHash: "IPFS_REPORT_123".
○ Click **Transact** and verify block execution.





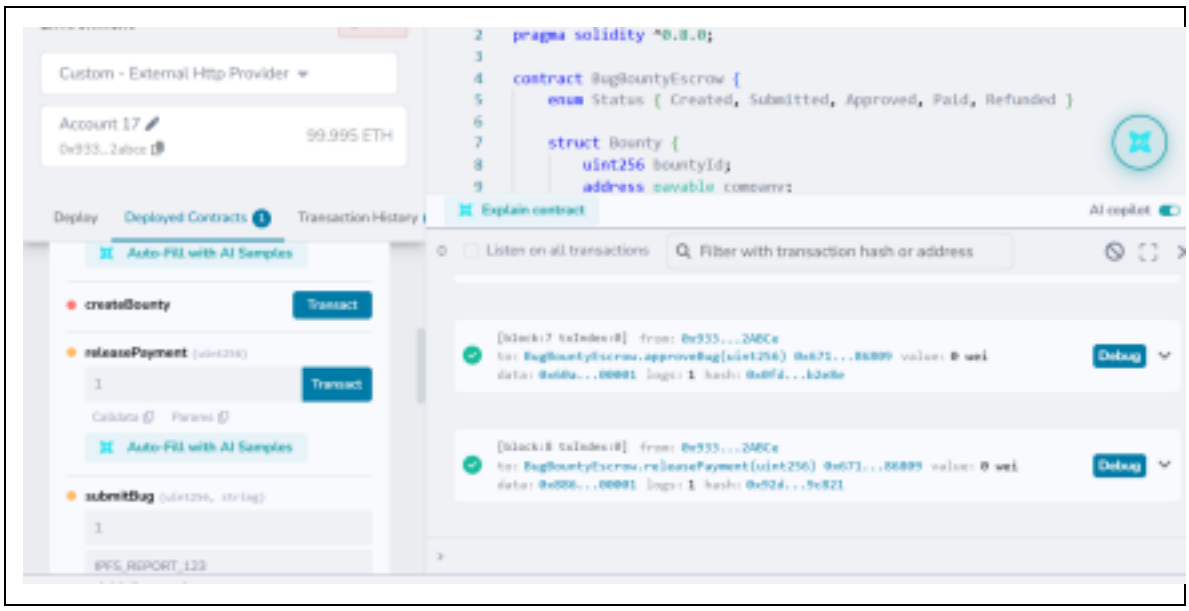
3. **approveBug**:
○ Expand **approveBug**, enter _bountyId: 1, and click **Transact**.
○ Verify the status update in the terminal log.



4. **releasePayment**:

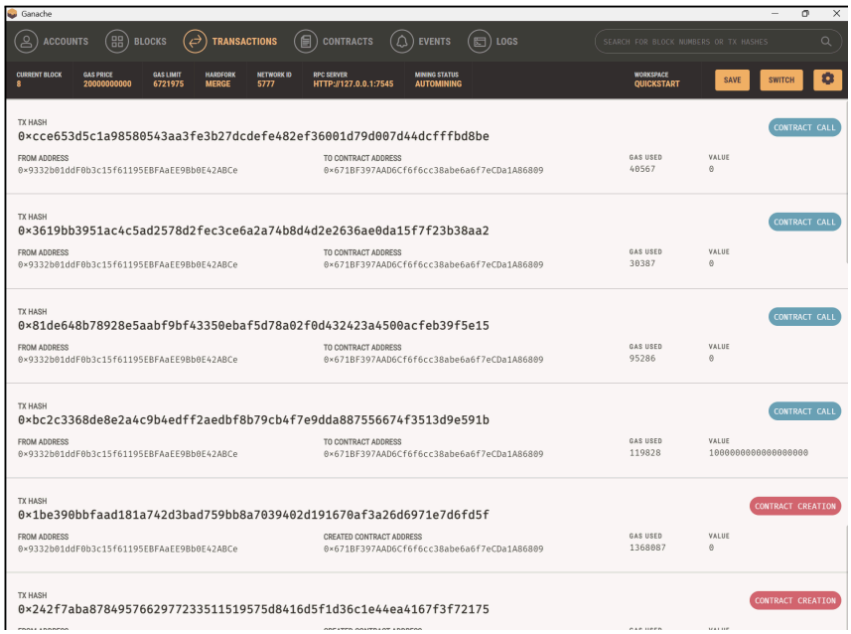
○ Expand **releasePayment**, enter _bountyId: 1, and click **Transact**.

○ Confirm that the locked 1 ETH reward is successfully transferred to the hunter account.



Step 5: Verification & Logging

1. Open the **Ganache Desktop GUI** and navigate to the **TRANSACTIONS** tab.
2. Verify all mined blocks (from deployment to final release) along with gas used, transaction hashes, and updated account ETH balances.
3. Capture screenshots of the transaction history for documentation.



Conclusion: Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with Metamask and Remix, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.